第 8 章 随机位生成和流密码

0. 学习目标总览

教材给出本章六个学习目标：

1. 解释随机数的随机性概念及其不可预测性。
2. 理解真随机数生成器、伪随机数生成器和伪随机函数之间的差异。
3. 简要介绍伪随机数生成器的要求。
4. 解释如何用分组密码构建伪随机数生成器。
5. 简要介绍流密码和 RC4。
6. 解释偏差的含义。

随机位流用于密钥分发、相互认证中的时变值、会话密钥、RSA 密钥生成和对称流加密。本章将确定性生成随机位的 PRNG/DRBG 与物理熵源产生随机位的 TRNG/NRBG 并列讨论，并说明二者可组合为既高效又具不可预测性的生成机制。

```
随机位生成
  |
  ├── 随机性
  |      ├── 分布均匀性
  |      `-- 独立性
  |
  ├── 不可预测性
  |      ├── 正向：不可预测下一位
  |      `-- 反向：不可由输出推回种子
  |
  ├── 确定性生成
  |      ├── PRNG：不限长位流
  |      ├── PRF：固定长输出
  |      `-- CTR_DRBG、OFB、BBS
  |
  ├── 流密码
  |      ├── RC4
  |      `-- LFSR、NFSR、Grain-128a
  |
  `-- 非确定性生成
         ├── 熵源
         ├── 调节：校正偏差、提升熵
         `-- 健康测试与英特尔 DRNG
```

1. 学习目标 1：解释随机性与不可预测性

1.1 随机数的用途

教材列出的密码与网络安全用途包括：

1. 密钥分发与相互认证协议中的时变值，用于抵抗重放攻击；
2. 会话密钥生成；
3. RSA 公钥加密的密钥生成；
4. 对称流加密的密钥流生成。

这些应用提出两项不同但未必相容的要求：**随机性**与**不可预测性**。

1.2 随机性的两个统计标准

教材从统计角度给出两个随机性评价标准：

标准	含义
分布均匀性	位序列中 0 和 1 出现的频率大致相等。
独立性	序列的任意子序列不能由其他子序列推导。

可以用成熟测试检验是否符合特定分布，例如均匀分布；但没有测试能证明序列独立。测试只能发现不独立性，因此实践策略是使用多个测试，若它们都未发现不独立性，则对独立性建立较强信心。

NIST SP 800-22 的随机性测试应具有三项性质：

性质	教材说明
均匀性	任意位置上 0、1 出现概率均应接近 $1/2$。
可伸缩性	对完整序列有效的测试也应对随机抽取的子序列有效。
一致性	对所有种子，生成器性质都应一致；单一种子测试不足够。

教材列举的测试例子有频率测试、行程测试和 Maurer 通用统计测试：后者考察模式匹配间隔及可压缩性，能被大幅压缩的序列被视为非随机。

1.3 不可预测性

在认证、会话密钥与流密码中，序列不仅应统计随机，还必须使后续值不可预测。伪随机位流需要两种不可预测性：

类型	含义
正向不可预测性	不知道种子时，即使知道前面任意多位，也不能预测下一位。
反向不可预测性	不能由生成的任何值推断种子。

真正随机序列中的成员统计独立，因而不可预测；但真随机数速率和效率有限。确定性生成的伪随机数虽不是真随机，却必须阻止敌手由已见序列推导之后的成员。

2. 学习目标 2：理解 TRNG、PRNG 和 PRF 的区别

2.1 三类生成机制

名称	输入与机制	输出与用途	关键特点
TRNG / NRBG	物理熵源及其数字化、可能的调节	非确定随机位	**非确定**、低效、无周期
PRNG / DRBG	种子与确定性算法	**不限长度**伪随机位流	高效、**确定**、通常有很长周期
PRF	种子与上下文相关输入	**固定长度**伪随机位串	用于对称密钥、时变值等

TRNG 的熵源可包括键盘敲击时间、磁盘活动、鼠标移动、系统时钟的瞬时值等；它也可能只做模拟源到二进制输出的转换，或进一步消除源的偏差。

PRNG 以固定种子为输入，并以反馈方式用确定性算法产生位流。因此，知道算法和种子的敌手可重现全部输出。PRNG 与 PRF 在生成长度外没有本质差别，两者都需种子，都需随机性与不可预测性；本章后续不严格区分二者。

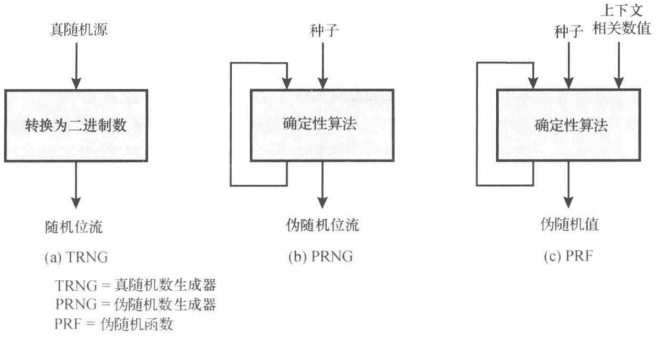


图 8.1　随机和伪随机数生成器

图 8.1 TRNG 与两类伪随机数生成器

2.2 PRNG 种子要求与 TRNG 的位置

PRNG 是确定性算法，所以种子若被推导，输出即被确定。种子必须不可预测，本身应是随机数或伪随机数。SP 800-90A 推荐由 TRNG 产生种子后输入 PRNG。

TRNG 不直接替代 PRNG 的原因是：流密码需与消息等长的密钥流，若直接使用 TRNG，就必须安全分发整条密钥流；PRNG 只需安全共享较短流密码密钥。TRNG 输出还可能含偏差，PRF 的输出可对其调节。最后，TRNG 的生成速率未必满足应用的随机位需求。

3. 学习目标 3：简要介绍 PRNG 的要求

3.1 随机性与不可预测性要求

密码应用要求不知道种子的敌手不能确定伪随机串。若 PRNG 输出不够随机，例如由 128 位种子和上下文生成的 128 位密钥呈现规律，敌手可能比穷举更容易地攻击该密钥。

因此，PRNG/PRF 的输出需要：通过足够的随机性测试、正向不可预测、反向不可预测，以及使用安全种子。随机测试不能证明随机性，却能发现不随机性；一致通过多个测试可作为工程

上的证据。

3.2 两类算法设计

教材将 PRNG 算法分为两类：

类型	特点
专用算法	专门生成伪随机位流；有的用于通用 PRNG，有的为流密码设计。
基于已有密码算法	利用分组密码（CTR）、哈希函数或消息认证码对输入的随机化能力。

SP 800-90A 推荐以对称分组密码、哈希函数、消息认证码构造强 PRNG。已使用加密或认证算法的系统可以复用这些实现。

3.3 线性同余生成器：统计可用但密码学不安全

线性同余生成器有模 m、乘数 a、增量 c、种子 X_0 四参数：

$$X_{n+1} = (aX_n + c) \bmod m$$

Park 与 Miller 给出的评价标准是：全周期、序列看似随机、可用 32 位算术有效实现。若 m 是素数、$c = 0$，可选取某些 a 得到周期 $m - 1$；常用 $m = 2^{31} - 1$、$a = 16807$。

即使参数合适，它仍不适合密码用途：给定种子即确定后续序列；敌手若知道算法和参数，看到一个随机数即可得到后续数；即便未知参数，少量连续输出也足以解出 a, c, m。它适用于统计和仿真，不提供密码学所需的不可预测性。

3.4 BBS：密码安全伪随机位生成器

BBS 选择大素数 p, q，满足

$$p \equiv q \equiv 3 \pmod 4$$

令 $n = pq$，选取与 n 互素的随机种子 s，然后迭代：

$$X_0 = s^2 \bmod n$$

$$X_i = X_{i-1}^2 \bmod n, \qquad B_i = X_i \bmod 2$$

每轮输出状态最低有效位。BBS 是 CSPRBG，即能通过下一位测试的生成器：不存在多项式时间算法能以高于 $1/2$ 的概率，从前 k 位预测第 $k+1$ 位。其**安全性基于对 n 分解为 p, q 的困难性**。

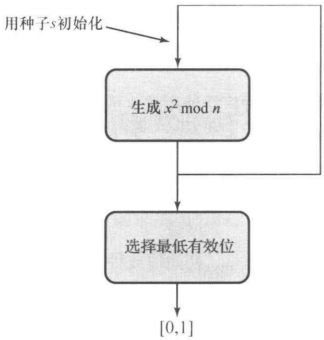


图 8.3　BBS 框图

图 8.3 BBS 生成器框图

4. 学习目标 4：用分组密码构建 PRNG

4.1 CTR 与 OFB 构造

对称分组密码的密文应呈现随机外观，因此适合用作 PRNG 的核心。CTR、OFB 是常见构造；SP 800-90A、ANSI X9.82、RFC 4086 推荐 CTR，后两者也推荐 OFB。

种子由密钥和状态值 V 构成。以 AES-128 为例，种子包含 128 位密钥和 128 位 V。每次生成一个 AES 的 128 位输出块：

模式	迭代规则
CTR-PRNG	$V = (V + 1) \bmod 2^{128}$，输出 $E(K, V)$。
OFB-PRNG	$V = E(K, V)$，输出 V。

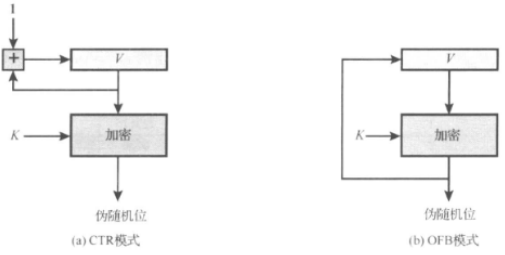


图 8.4　基于分组密码的 PRNG 机制

图 8.4 基于分组密码的 PRNG 机制

这两种生成器应在输出中呈现接近一半的 1 位，并且相邻块的匹配位比例接近 0.5，以表明块间无明显相关。

4.2 NIST CTR_DRBG

CTR_DRBG 使用 3DES 或 AES，相关参数如下：

参数	含义
outlen	加密算法输出分组长度。
keylen	加密密钥长度。
seedlen	种子长度，$\text{seedlen} = \text{outlen} + \text{keylen}$。
reseed_interval	使用新种子更新前允许输出的最大分组数。

初始化时，可用任意初始 K, V，以 CTR 方式生成至少 seedlen 位；再将输出最高 seedlen 位与熵源提供的 seedlen 位 XOR，形成新种子。新种子高 keylen 位作为新 K，低 outlen 位作为新 V。

生成阶段固定密钥、每块后递增 V，不断输出 $E(K, V)$。为限制使用同一状态输出的位数，补种计数器达到 reseed_interval 后调用更新函数（种子具有时效性，过期需更新）；它使用新熵位产生新的 (K, V)，然后继续生成。

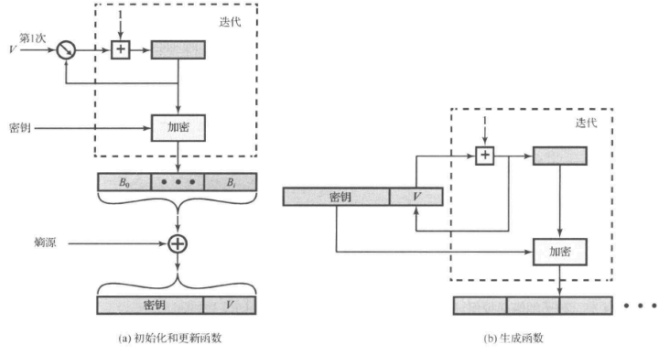


图 8.5 CTR_DRBG 函数

图 8.5 CTR_DRBG 的初始化、生成与更新函数

5. 学习目标 5：简要介绍流密码和 RC4

5.1 流密码的结构与安全要求

流密码可视为一次一密的伪随机版本：一次一密使用与消息等长的真随机密钥流；流密码用短密钥生成计算上等价于随机位流的伪随机密钥流。它适于高速流数据和内存、处理能力有限的受限设备。

典型流密码有：随时间演化的秘密状态 s_i、状态转换函数 f、输出函数 g 生成的密钥流 z_i。密钥 K 初始化状态，某些算法还使用 IV；IV 不必保密，但应不可预测且唯一。

加解密使用 XOR：

$$c_i = z_i \oplus m_i, \qquad m_i = z_i \oplus c_i$$

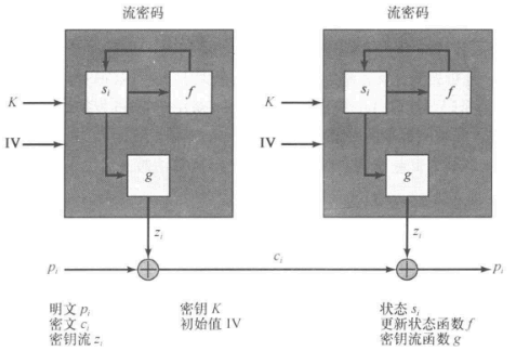


图 8.6　典型流密码的通用结构

图 8.6 典型流密码的通用结构

流密码设计应满足：密钥流周期长；密钥流尽量接近真随机（如 0、1 或各字节值频率接近均匀）；密钥足够长，教材给出当前技术下至少 128 位。相同密钥不能用于加密两个明文：两个密文 XOR 会得到两个原始明文的 XOR，从而极大简化密码分析。

5.2 RC4

RC4 是 Ron Rivest 于 1987 年设计的、面向字节运算的变长密钥流密码。它使用一个 256 字节状态向量 S；任一时刻 S 都是 0 至 255 的排列。每个输出字节后，状态继续重新置换。

初始化。 先置 $S[i]=i$，把密钥循环复制到临时向量 T：

```
for (i=0; i<256; i++) {
    S[i] = i;
    T[i] = K[i mod keylen];
}
```

再执行：

```
j = 0;
for (i = 0; i < 256; i++) {
    j = (j + S[i] + T[i]) mod 256;
    Swap(S[i], S[j]);
}
```

仅有交换操作，故最终 S 仍是 0 至 255 的排列。

流生成。 初始化后不再直接使用输入密钥；每次增加 i、更新 j、交换两个状态项，再选取一个输出字节：

```
i = 0;
j = 0;
while (1) {
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
```

```
    Swap(S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];
}
```

> 一般 `while (1)` 中的 1 为**需要密钥流字节**。

加密时 k 与下一明文字节 XOR，解密时与下一密文字节 XOR。教材指出，RC4 密钥调度存在基本漏洞，后续密码分析又利用密钥流偏差恢复重复加密的明文；RFC 7465 禁止 TLS 使用 RC4，NIST 也禁止其用于政府 TLS。

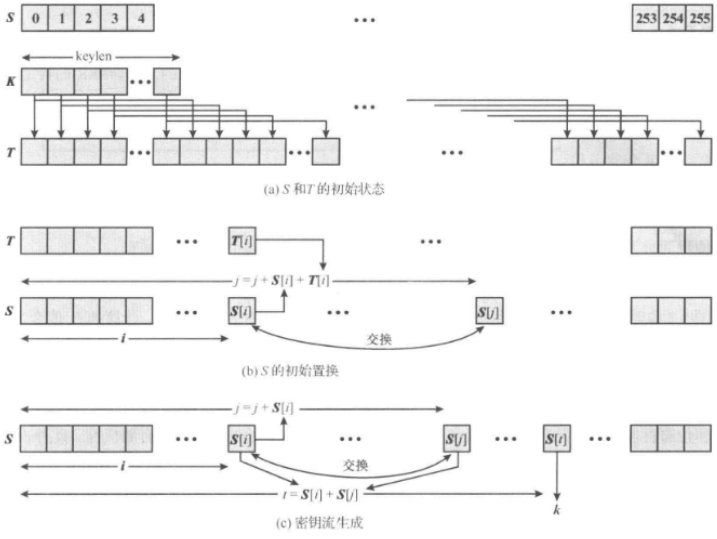


图 8.7　RC4

图 8.7 RC 4

6. 补充：反馈移位寄存器与 Grain-128a

6.1 LFSR 与 NFSR

反馈移位寄存器（FSR）由一串 1 位存储单元组成。每次时钟，最低位移出作为输出，其他位右移，并由其他位计算新的最高位。它适合紧凑、高效、低功耗硬件实现。

LFSR 的反馈仅使用模 2 加法（XOR）。一个 n 位 LFSR 可写为

$$B_n = \sum_{i=1}^{n} A_i B_{n-i} \tag{8.1}$$

其特征多项式为

$$P(X) = 1 + \sum_{i=1}^{n} A_i X^i \tag{8.3}$$

n 位 LFSR 最大周期为 $2^n - 1$。当特征多项式本原时，产生最大长度序列（m 序列）。LFSR 周期长、统计性质好、易于硬件实现，但单个 LFSR 不适合作为流密码：若反馈函数已知，知道连续 n 位即可确定全部流；若反馈函数未知，$2n$ 位足以确定全部流。

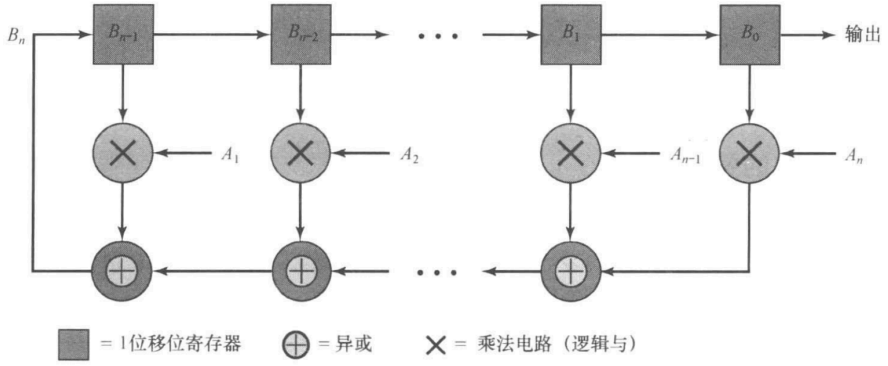
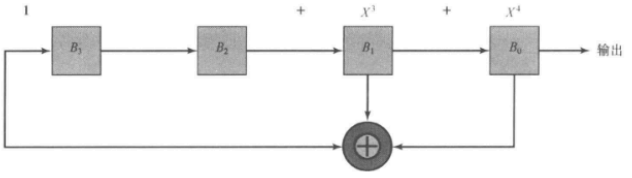


图 8.8　二进制线性反馈移位寄存器序列生成器

图 8.8 二进制线性反馈移位寄存器序列生成器



(a) 移位寄存器的构成

状态	B_3	B_2	B_1	B_0	$B_0 \oplus B_1$	输出
初始值 = 0	1	0	0	0	0	0
1	0	1	0	0	0	0
2	0	0	1	0	1	0
3	1	0	0	1	1	1
4	1	1	0	0	0	0
5	0	1	1	0	1	0
6	1	0	1	1	0	1
7	0	1	0	1	1	1
8	1	0	1	0	1	0
9	1	1	0	1	1	1
10	1	1	1	0	1	0
11	1	1	1	1	0	1
12	0	1	1	1	0	1
13	0	0	1	1	0	1
14	0	0	0	1	1	1
15=0	1	0	0	0	0	0

(b) 初始状态为1000的示例

图 8.9　4 位线性反馈移位寄存器

图 8.9 四位 LFSR 及其最大长度序列示例

NFSR 的反馈系数可以是变量，从而引入非线性，例如 $B_4 = B_0 \oplus B_1 B_3$。NFSR 本身也缺少可分析其安全性的理论；将 LFSR 与 NFSR 组合可以同时获得已知最大周期和高安全性。

6.2 Grain-128a

Grain-128a 是硬件高效流密码，由一个 128 位 LFSR、一个 128 位 NFSR 和滤波函数组成。LFSR 保证最短周期与输出平衡性；NFSR 和非线性滤波器提供非线性；LFSR 输出掩蔽 NFSR 输入，使 NFSR 状态保持平衡。

其 LFSR 反馈为

$$s_{i+128} = s_i \oplus s_{i+7} \oplus s_{i+38} \oplus s_{i+70} \oplus s_{i+81} \oplus s_{i+96}$$

输出函数取跳过前 64 位后每隔一位的预输出：

$$z_i = y_{64+2i}$$

128 位密钥初始化 NFSR；96 位 IV 及固定填充值初始化 LFSR。随后两个寄存器时钟 256 次而不输出密钥流，预输出反馈参与寄存器输入，以置乱初始内容。消息加解密仍为 $c_i = z_i \oplus m_i$、$m_i = z_i \oplus c_i$。Grain-128a 还用未参与加密的输出位和 32 位累加器生成 32 位认证标记。

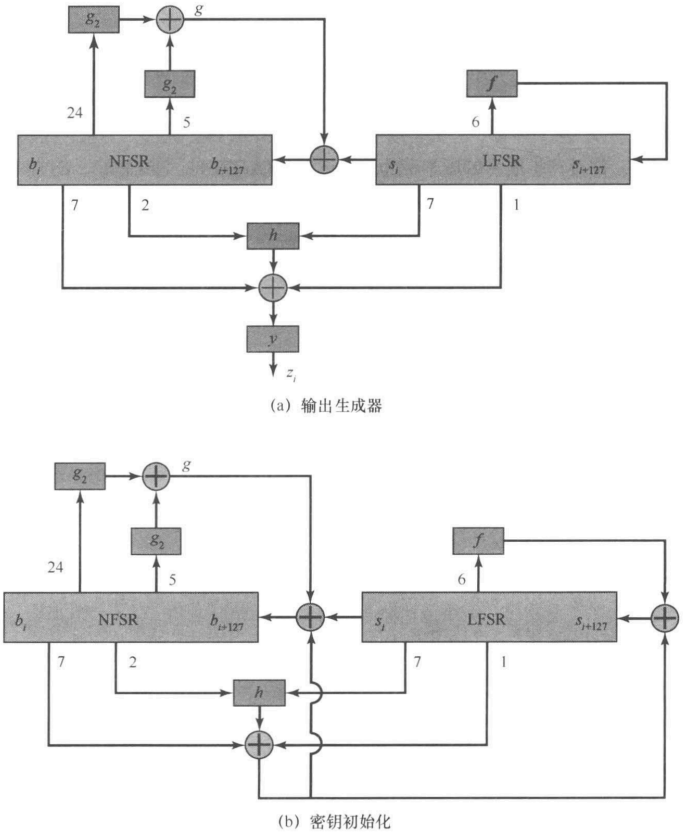


图 8.12　Grain-128a 流密码

图 8.12 Grain-128a 的输出生成与密钥初始化

7. 学习目标 6：解释偏差的含义

7.1 熵源、偏差与熵率

TRNG 使用不确定物理源获取随机性，例如电离辐射、气体放电、泄漏电容器、热噪声、音频/视频输入、磁盘寻道时间波动等。相较 PRNG，TRNG 生成慢、非确定、非周期；PRNG 快、确定、通常周期极长。

偏差指某些潜在输出结果的出现概率大于其他结果。例如 TRNG 输出中 1 的数量可能偏多，或物理噪声叠加规则波、周期现象后看似随机却能被统计测试判定为确定。

熵率是熵源提供熵的速率，等于评估的熵位数除以输出总位数，取值在 0 至 1；对长度 n、最小熵 k 的随机源，熵率为 k/n。无偏、各位和位组独立的位流熵率为 1。

7.2 调节

调节又称偏差校正，使用密码算法置乱随机位，以消除偏差并提高熵。常见方法是哈希函数和对称分组密码。

方法	教材说明
哈希调节	将 $m \geq n$ 个输入位通过哈希函数，取 n 位输出；连续输入块形成连续输出块。（看图 8.2）
`hash_df`	以计数器、要求输出位数和输入串为哈希输入；连接多个哈希输出后取要求的最高位，最大输出不超过哈希输出长度的 255 倍。
分组密码调节	将 TRNG 位按块以 AES 等分组密码加密；英特尔实现使用 AES/CBC 形式的 CMAC 调节。

> **关于 哈希调节 理解**
>
> 在随机数生成体系中，哈希函数常作为调节算法，将 TRNG 输出的原始数据（输入 m 位分组）压缩并转换成 PRNG 所需的定长种子（输出 n 位），其核心目的是利用哈希函数的雪崩效应来平滑物理噪声中存在的统计偏差，从而为后续的确定性扩展生成提供均匀且高质量的初始状态，但需要注意这一过程并不会增加实际的物理熵值，且通过要求输入长度 m 大于输出长度 n 来确保有足够的熵余量。

7.3 健康测试

不确定随机位生成器的一般模型是：硬件噪声源产生随机输出，数字化为真位源；调节模块减少偏差、最大化熵；健康测试分别监控数字化输出和调节器输出。

SP 800-90B 为噪声源建议：

测试	目的
重复计数测试	检测噪声源长期卡在一个值的灾难性故障。
自适应比例测试	在连续样本窗口内检测某样本过于频繁，从而发现大的熵损失。

最小熵为 H 位/样本时，连续 C 个相同样本的概率满足

$$\Pr[\text{连续 } C \text{ 个相同样本}] \le (2^{-H})^{C-1}$$

若可接受假阳性概率为 W，重复计数截止值为

$$C = \left\lceil 1 + \frac{-\log W}{H} \right\rceil$$

对调节函数输出进行健康测试的目标是确认其表现为真随机位流，合理做法是采用 SP 800-22 的随机性测试。

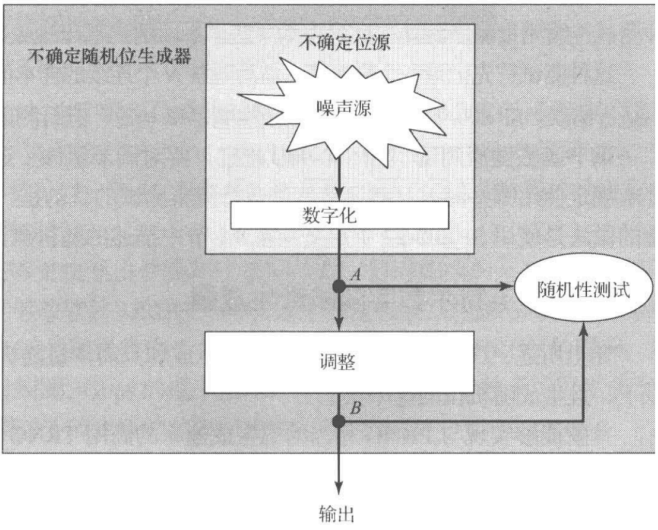


图 8.13　NRBG 模型

图 8.13 NRBG 的噪声源、调节与健康测试模型

7.4 英特尔数字随机数生成器

英特尔 DRNG 于 2012 年在多核芯片中部署，特点是全硬件实现且与处理器同芯片，减少 I/O 延迟。其三阶段结构为：

1. 两个相互馈电的反相器在不稳定状态下受热噪声驱动，产生随机位；该阶段可达 4 Gbps；
2. 每次收集 512 位，用 CBC-MAC/CMAC 调节，输出 256 位无偏真随机性；
3. 以 256 位调节结果作种子，使用 CTR_DRBG 高速生成伪随机数；每个种子可生成 511 个 128 位样本。

PRNG 阶段通过 `RDRAND` 提供 16、32、64 位随机值；调节器输出还可由增强型不确定随机数生成器提供为密码种子，并通过 `RDSEED` 读取。

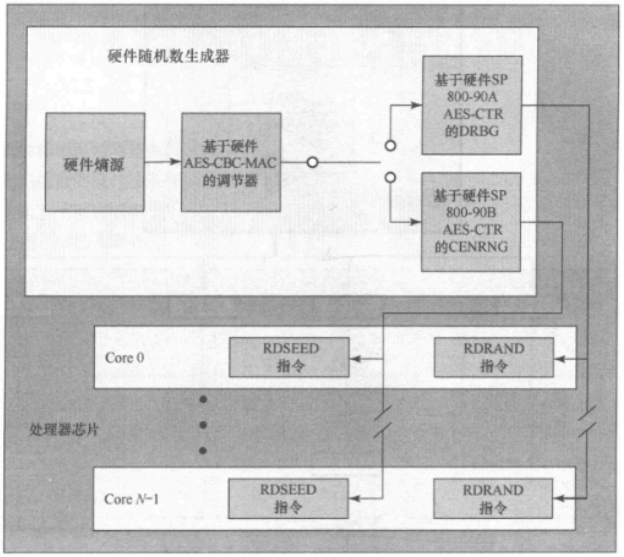


图 8.14　DRNG 的整体结构

图 8.14 英特尔 DRNG 整体结构

本章关键术语整理

术语	教材含义	
随机性	具有分布均匀性和独立性的统计性质。	
不可预测性	不能由已知输出预测后续输出或推回种子。	
TRNG / NRBG	基于物理熵源的非确定随机位生成器。	
PRNG / DRBG	以种子和确定性算法产生伪随机位的生成器。	
PRF	生成固定长度伪随机位串的函数。	
种子	决定确定性 PRNG 输出状态的安全输入。	
CSPRBG	能通过下一位测试的密码安全伪随机位生成器。	
CTR_DRBG	以计数器模式分组密码构建的确定性随机位生成器。	
流密码	以伪随机密钥流和明文 XOR 进行加解密的密码。	
LFSR / NFSR	线性 / 非线性反馈移位寄存器。	
偏差	熵源各可能结果并非等概率出现的现象。	
熵率	熵源每输出位提供的熵量，取值在 0 到 1。	
调节	用密码算法校正偏差、提高熵质量的过程。	

学习目标自检

1. 能区分统计随机性的均匀性、独立性与密码应用所需的不可预测性。
2. 能解释正向、反向不可预测性，并说明种子为何必须不可预测。

3. 能比较 TRNG、PRNG、PRF 的输入、确定性、输出长度与典型用途。
4. 能说明线性同余生成器为什么可用于统计仿真却不适用于密码学。
5. 能写出 BBS 的初始化、平方迭代和最低位输出，并说明其安全性基础。
6. 能说明 CTR-PRNG 与 OFB-PRNG 的状态更新区别，以及 CTR_DRBG 的初始化、生成、补种过程。
7. 能描述典型流密码的状态、状态转换函数、输出函数、密钥和 IV 的关系。
8. 能说明流密码不能重用同一密钥的原因，并概述 RC4 的状态初始化和流生成。
9. 能说明 LFSR 最大周期、其作为单独流密码的不安全性，以及 LFSR 与 NFSR 结合的动机。
10. 能解释偏差、熵率、调节和两类噪声源健康测试的作用。
11. 能概括英特尔 DRNG 的热噪声、CMAC 调节、CTR_DRBG 三阶段结构，以及 RDRAND、RDSEED 的用途。